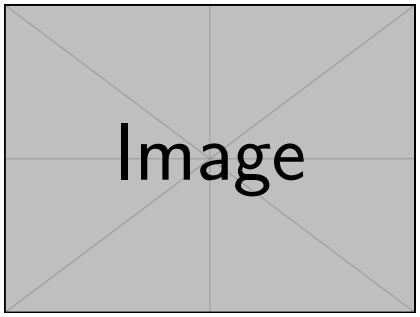

A DNA-BERT Architecture for Promoter and Enhancer Classification

...

Garagnani Filippo, Veronese Alex

Dipartimento di Ingegneria 'Enzo Ferrari', Modena, Italy

Università degli Studi di Modena e Reggio Emilia

May 27, 2026

Supervised by: Dott. Lovino Marta *Course:* AI For Bioinformatics

Abstract

Promoters are regulatory DNA regions, usually located upstream of genes, that control the initiation of transcription by providing binding sites for RNA polymerase and transcription factors. Enhancers are distal cis-regulatory elements that modulate gene expression by increasing transcriptional activity, often acting over long genomic distances through complex chromatin interactions with promoters. Accurate identification of promoters and enhancers is a fundamental problem in computational genomics, as these elements play a crucial role in gene regulation, cellular differentiation, and disease mechanisms. However, the precise location and functional boundaries of regulatory elements within the genome are often difficult to determine experimentally, making computational prediction methods particularly valuable. Traditional bioinformatics approaches rely heavily on handcrafted sequence features and motif discovery techniques, which may fail to capture the complex contextual dependencies present in genomic sequences. In this project, we adapt the DNA-BERT architecture to the tasks of promoter and enhancer classification. By leveraging transformer-based language modeling techniques originally developed for natural language processing, DNABERT-2 is able to learn rich contextual representations of nucleotide sequences directly from raw DNA data. We investigate the effectiveness of this approach for distinguishing regulatory elements and evaluate its performance on genomic datasets.

Keywords: deep learning, BERT, promoter prediction, enhancer prediction, bioinformatics, transformer

Contents

1 Introduction

The rapid advancement of high-throughput sequencing technologies has produced an unprecedented volume of genomic data. Identifying functional elements, such as promoters, directly from raw DNA sequence is a fundamental challenge in bioinformatics, particularly because they are not defined structurally [9]. Traditional methods rely on conserved motifs (e.g., TATA box, GC-box), but these motifs are often degenerate and fail to capture long-range dependencies.

Recent breakthroughs in natural language processing (NLP), particularly the Transformer architecture and the BERT model, have inspired sequence-based models in genomics. DNA can be viewed as a language with its own alphabet $\Sigma = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$. One of the principal advantages of attention-based architectures is their ability to efficiently model interactions between distant elements within a sequence, a property that is required for a wide range of sequence analysis tasks. This project investigates how a DNA-specific BERT model behaves in a binary classification scenario of promoter versus enhancer sequences.

1.1 Motivation

A precise computational promoter and enhancer predictor can drastically reduce the experimental effort required for gene annotation and can uncover novel regulatory mechanisms. Machine learning, and especially deep learning, offers the possibility to autonomously learn complex sequence features without hand-crafted rules. ...

1.2 Objectives

- ...

2 Background

2.1 Biological Foundations

A **promoter** is a DNA region typically located upstream of a gene's transcription start site (TSS) that plays a central role in initiating transcription. RNA polymerase II binds to the core promoter with the assistance of general transcription factors, forming the pre-initiation complex required for mRNA synthesis. Promoters are essential for defining where transcription begins and are often associated with gene-specific regulatory programs and cell-type-specific expression levels.

An **enhancer** is a DNA regulatory element that can increase transcriptional activity of target genes, often acting independently of its orientation and at variable distances from the gene it regulates. Unlike promoters, enhancers do not directly recruit RNA polymerase II but instead function by interacting with promoters through chromatin looping and the recruitment of transcriptional co-activators and transcription factors. Enhancers can reside upstream, downstream, or within intronic regions of genes, and their activity is highly context-dependent, varying across cell types, developmental stages, and environmental conditions.

2.2 The Transformer

The Transformer [10] eschews recurrence and convolution operations in favour of **self-attention**, enabling parallelisation and the capture of long-range dependencies.

Attention is a sequence-to-sequence operation that mainly outputs elements in the input sequence into an updated version of them. More specifically, given an input sequence

$X \in \mathbb{R}^{n \times d}$, we project each element through three different mappings:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

Semantically, what these three projections – called *query*, *key* and *value* – are doing, is to extract from each vector in the sequence, respectively, what an element is looking for, what an element is offering and what an element is actually containing. The main attention operation:

$$A = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$$

then, computes pairwise similarity between each couple of elements ($Q \cdot K^T$), converting the raw similarities values into a probability distribution (softmax$(\cdot)$) and then computing a weighted average of the vector values, using the similarity values as weights. The fact that in a single computation each element is immediately compared with every other element is one of the main reasons of the efficiency of the Transformer architecture, unlike Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs).

2.3 BERT

BERT [2] pre-trains a bidirectional Transformer using masked language modelling (MLM) and next-sentence prediction. MLM is a task that requires a language model to predict the nature of a set of masked elements in a sequence. Formally, given a sequence $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, each element is masked with probability p; it means that an element gets swapped for a made-up 'unknown' element $[MASK]$. The model is then trained to reconstruct the original token at the masked positions by leveraging both left and right context simultaneously, which forces it to learn bidirectional representations of the input sequence.

This self-supervised learning approach is successfully used to create meaningful internal representations of the possible elements in the sequence. This semantic embeddings can then be employed to perform supervised learning tasks on it by training an ad-hoc additional classification head.

2.4 DNABERT-2

DNABERT-2 [11] represents a major leap compared to the severe computational inefficiencies that bottlenecked its predecessor. While the original DNA-BERT model relied on overlapping k-mer tokenization [5] – a method that generates fixed-length permutations of nucleotides and has a few downsides – DNABERT-2 introduces Byte Pair Encoding (BPE) to the genomic space. Moreover, it employs ALiBi for positional encodings [8], enabling the model to deal during evaluation with input sequences longer than those seen during training.

Byte Pair Encoding. A standard technique for tokenizing genomic sequences is the k-mer method, which generates all possible contiguous sequences of k nucleotides. For instance, with $k = 3$, the sequence `ACGT` would be tokenized into `ACG` and `CGT`. This approach, however, has several limitations. First of all, it presents a computational bottleneck, since the number of possible tokens is exponential in k. Then, for masked language modeling tasks, masking a single token is not enough, since the model could easily infer the masked token from the unmasked neighbouring ones – meaning that k-mer need to be masked contigously.

DNABERT-2, instead, uses Byte Pair Encoding (BPE), a data compression technique that iteratively determines the most frequent tokens. The approach works as follows: starting with a base vocabulary of single nucleotides, the algorithm identifies the most frequent pair of tokens in the training corpus and merges them into a new token. This process is repeated until a predefined vocabulary size is reached, resulting in a set of tokens that can represent common sequences more efficiently and with a smaller vocabulary than fixed-length k-mers.

ALiBi. The attention function is inherently position-agnostic – a permutation on the input elements in the sequence would produce the same output, albeit with a different order. To overcome this problem, the original Transformer architecture introduced positional encodings, which are added to the input embeddings to inject information about the position of each token in the sequence. However, these fixed positional encodings limit the model's ability to generalize to longer sequences than those seen during training.

ALiBi (Attention with Linear Biases) is a method that modifies the attention mechanism to include a bias term that linearly increases with the distance between tokens. Formally, given a sequence of length n, the attention score between tokens at positions i and j is modified as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} - m_h|i-j|\right)V$$

with m_h a parameter that is head-specific and is fixed before training. Intuitively this bias encourages the model to attend more to nearby tokens, while still allowing it to capture long-range dependencies when necessary. Moreover, as seen, unlike learned positional encodings, ALiBi does not require any additional learnable parameters.

3 Related Works

DNA-BERT. The original DNA-BERT model [5] was one of the first to apply the BERT architecture to genomic sequences, using k-mer tokenization. It demonstrated that transformer-based models could learn meaningful representations of DNA sequences and achieve competitive performance on various genomic tasks, including promoter prediction.

DNA-BERT2. DNABERT-2 [11] builds upon the original DNA-BERT by introducing BPE tokenization and ALiBi positional encodings, which significantly improve the model's efficiency and ability to handle longer sequences. The model was pre-trained on a large corpus of genomic sequences from multiple species and evaluated on a variety of downstream tasks, showing superior performance compared to its predecessor and other baseline models.

Nucleotide Transformer. ...

EVO. ...

4 Methods

4.1 Data Collection and Preprocessing

To construct the foundational sequence library for analysis, primary genomic assemblies and regulatory coordinates for Homo sapiens were compiled from several public repositories.

Human Genome. The complete reference genome sequence for Homo sapiens (Assembly version GRCh38) was acquired via the Ensembl FTP server [4]. To facilitate modular processing and preserve chromosomal segmentation, files were fetched individually for each human chromosome ($1 \leq \text{num} \leq 22$) in a compressed FASTA layout. To isolate functional genomic regions, comprehensive gene feature annotations were still retrieved from the Ensembl FTP Server. The comprehensive Gene Transfer Format (GTF) file containing coordinate structures was then used in order to extract another FASTA file layout containing coding sequences, filtering out every non-coding portion of each chromosome (e.g., introns).

Promoters and Enhancers. In order to later apply our method to promoter and enhancer classification, the data has been collected from public sources. More specifically, the Eukaryotic Promoter Database (EPDNew) [3] was employed to retrieve the data relative to the human promoters. Additionally, from ... (add here the source of the enhancer data) the enhancer sequences were obtained.

4.2 Model Architecture

The model architecture is based on the DNABERT-2 framework, which consists of a multi-layer bidirectional Transformer encoder.

Input. The input to the model is a tokenized DNA sequence, where tokens are generated using Byte Pair Encoding (BPE) (see Section 2). The token vocabulary, consisting of $N = 4096$ distinct tokens, was retrieved from the original DNABERT-2 implementation. Each token gets converted to a learnable embedding vector of dimension $d = 768$, which serves as the input to the Transformer layers.

The Encoder. Each Encoder Module consists of a multi-head self-attention mechanism, followed by a feed-forward network. After each sub-layer, residual connections and layer normalization are applied. Specifically, each self-attention counts $h = 12$ heads; the feed-forward network has an hidden dimensionality of $d_{ff} = 3072$.

The Encoder, the core component of the architecture, is a stack of $L = 12$ identical layers of the aforementioned Encoder Module. The output of the final encoder layer is a contextualized representation of the input sequence, which captures both local and long-range dependencies among the tokens. It can be hence used to further perform downstream tasks, such as promoter and enhancer classification, by adding a simple classification head on top of it.

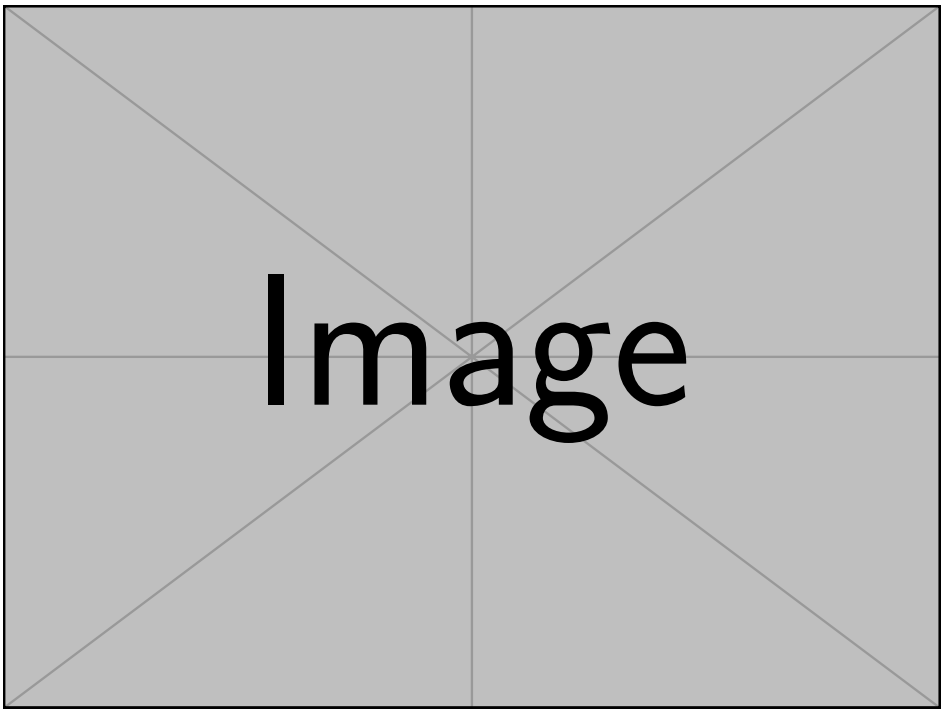

Figure 1: Architecture overview: DNABERT fine-tuned for promoter prediction.

4.3 Training Protocol

Rather than directly fine-tuning a publicly available DNABERT-2 checkpoint, we reimplemented the architecture and conducted domain-specific pretraining on coding DNA sequences before fine-tuning the model for promoter vs enhancer classification.

Pretraining. In order to give the architecture the capabilities of managing the complex patterns inherent in genomic data, a pretraining phase is conducted on a large-scale dataset of unlabeled DNA sequences. More specifically, in order to enhance its ability to understand the underlying structure behind the genome, the corpus used was entirely made up of only coding sequences, which are the most informative ones in terms of regulatory elements.

The pretraining objective is masked language modeling (MLM), where a certain percentage of tokens in the input sequence are randomly masked, and the model is trained to predict the original tokens based on the context provided by the unmasked tokens. The prediction head is a stack of two components: the *Transformation Layer*, comprising a single linear layer that does not change the dimensionality of the data, followed by a GELU and a layer normalization; and the *Decoder Layer*, which is a linear layer that maps the output of the transformation layer to the size of the token vocabulary, producing a probability distribution over possible tokens for each masked position. More specifically, the CDS data is collected and split into training and testing sets. The split happens at the chromosome level in order to avoid data leakage, meaning that the model is tested on sequences from chromosomes that were not seen during training.

The model is trained for 40000 steps. The learning rate is set to 1×10^{-3}, with 1200 warmup steps and a weight decay of 1×10^{-5}. The batch size is set to 8, and the masking probability is set to 15%. The optimizer used is AdamW [7], which is a variant of the Adam optimizer [6].

After the pretraining stage, only the weights of the encoder are kept, while the prediction head is discarded.

Fine-tuning. The fine-tuning phase involves adapting the pre-trained encoder to the specific downstream task of promoter and enhancer prediction. The model is trained on a balanced dataset of promoter and enhancer sequences, still with a chromosome-based splitting to avoid data leakage.

On top of the encoder, another classification head is added. This consists of a feedforward network, with hidden size 3072 and with GELU as the activation function, and followed by a final layer with output dimension 2 – one for each possible class.

The dataset contains promoters that are exactly 2000 nucleotides long, while enhancers have variable length, sometimes even exceeding the model's input length. If not handled, other than the length discrepancy, this could have lead the model to bypass the actual genomic semantics and simply learn to classify sequences based on their length. Moreover, it was not possible to trim the sequences without risking to lose important information, since the functional boundaries of promoters and enhancers are not well defined. To overcome all these problems, the following strategy was adopted: each input sequence is split into non-overlapping segments of at most 2000 nucleotides; each segment is passed through the model and then the resulting representations are mean-pooled together to produce a single representation for the whole sequence, which is then fed to the classification head. Formally, given an input sequence S of length L, we split it into $m = \lceil L/2000 \rceil$ non-overlapping segments $S_1, S_2, \ldots, S_m$. Each segment is processed independently through the encoder to obtain its contextualized representation: $S'_i = \text{Encoder}(S_i)$. Each segment S_i is itself a sequence of token (or position) embeddings produced by the encoder. To obtain a fixed-size vector for a segment we first perform intra-segment pooling (mean pooling over the token

dimension) to get a single vector v_i for each segment:

$$v_i = \frac{1}{|S_i|} \sum_{t=1}^{|S_i|} \text{Encoder}(S_i)_t,$$

where $|S_i|$ is the number of tokens in segment S_i. The segment vectors v_i are then aggregated via inter-segment pooling (mean across segments) to form the final sequence representation:

$$S' = \frac{1}{m} \sum_{i=1}^{m} v_i, \quad \hat{y} = \text{CLSHead}(S').$$

The model is fine-tuned for 4000 steps, with a learning rate of 1×10^{-3}, a batch size of 4 and a weight decay of 1×10^{-5}. During training, both the encoder and the classification head are updated. As the loss function, cross-entropy loss is used, which is appropriate for binary classification tasks.

4.4 Evaluation Metrics

To assess the performance of the model, a few evaluation metrics are employed.

Accuracy. Accuracy is the proportion of correctly classified samples out of the total number of samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is the number of true positives, TN is the number of true negatives, FP is the number of false positives, and FN is the number of false negatives. Assume, without loss of generality, that the positive class is the promoter class.

F1 Score. The F1 score is the harmonic mean of precision and recall, providing a balance between the two:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

where precision is the ratio of true positives to the sum of true positives and false positives, and recall is the ratio of true positives to the sum of true positives and false negatives:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

ROC-AUC. The Receiver Operating Characteristic Area Under the Curve (ROC-AUC) is a metric that evaluates the model's ability to discriminate between classes across different classification thresholds [1]. It is calculated by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings and computing the area under this curve:

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(t)\, d\text{FPR}(t)$$

where t represents the classification threshold. A higher ROC-AUC value indicates better model performance in distinguishing between promoters and enhancers.

5 Results

Two different models have been mainly evaluated. The first one has been a model pretrained on coding sequences only and later finetuned for the promoter vs enhancer classification task. The second model has been a model purely trained from scratch on the promoter vs enhancer classification.

5.1 Evaluation on Human Genome

In order to estimate the model's capabilities, the following evaluation procedure has been carrried out: each evaluated model was trained on 10-fold cross-validation; meaning, the whole dataset has been divided into 10 folds, 1 has been kept out for testing, and the model was trained on the remaining 9 folds. This has been done for $n = 7$ different random seeds to estabilish the model's robustness against different training dynamics and weights initialization. In total, we gathered $7 \times 10 = 70$ values both for the F1 score and for the ROC-AUC. Of course, cross-fold evaluations on the same seeds are not indipendent – two folds will never share the same sample to evaluate upon – so the 10 per-fold values within each seed were averaged into a single seed-level mean prior to any statistical estimation. This yields 7 independent observations per model, which form the basis for all reported confidence intervals.

Confidence intervals were estimated using a classical t-interval with $df = 6$, preferred over bootstrap at this sample size because the bootstrap distribution over $n = 7$ points is effectively discrete and underestimates tail uncertainty. All intervals are at the 95% level. Per-model results are reported in Table 1.

Table 1: Per-model performance on the human genome. Mean and 95% t-interval over 7 seed-level means (10-fold CV each).

Model	F1	F1 95% CI	ROC-AUC	ROC-AUC 95% CI
Pretrained + finetuned	0.684	(0.679, 0.689)	0.777	(0.774, 0.781)
From scratch	**0.688**	(0.679, 0.697)	**0.780**	(0.778, 0.782)

5.2 Evaluation on Mice Genome

...

6 Discussion

...

6.1 Limitations

...

7 Conclusion & Future Work

...

References

[1] Andrew P. Bradley. "The use of the area under the ROC curve in the evaluation of machine learning algorithms". In: *Pattern Recognition* 30.7 (1997), pp. 1145–1159. ISSN: 0031-3203. DOI: https://doi.org/10.1016/S0031-3203(96)00142-2. URL: https://www.sciencedirect.com/science/article/pii/S0031320396001422.

[2] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of NAACL-HLT*. 2019.

[3] René Dreos et al. "The Eukaryotic Promoter Database: expansion of EPDnew and new promoter analysis tools". In: *Nucleic Acids Research* 43.D1 (Jan. 2015), pp. D92–D96. ISSN: 0305-1048. DOI: 10.1093/nar/gku1111. eprint: https://academic.oup.com/nar/article-pdf/43/D1/D92/7318520/gku1111.pdf. URL: https://doi.org/10.1093/nar/gku1111.

[4] Sarah C Dyer et al. "Ensembl 2025". In: *Nucleic Acids Research* 53.D1 (Jan. 2025), pp. D948–D957. ISSN: 1362-4962. DOI: 10.1093/nar/gkae1071. eprint: https://academic.oup.com/nar/article-pdf/53/D1/D948/60949583/gkae1071.pdf. URL: https://doi.org/10.1093/nar/gkae1071.

[5] Yanrong Ji et al. "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome". In: *Bioinformatics* 37.15 (Aug. 2021), pp. 2112–2120. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btab083. eprint: https://academic.oup.com/bioinformatics/article-pdf/37/15/2112/57195892/btab083.pdf. URL: https://doi.org/10.1093/bioinformatics/btab083.

[6] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[7] Ilya Loshchilov and Frank Hutter. "Decoupled weight decay regularization". In: *arXiv preprint arXiv:1711.05101* (2017).

[8] Ofir Press, Noah A Smith, and Mike Lewis. "Train short, test long: Attention with linear biases enables input length extrapolation". In: *arXiv preprint arXiv:2108.12409* (2021).

[9] Stefan Rombauts et al. "Computational approaches to identify promoters and cis-regulatory elements in plant genomes". In: *Plant Physiology* 132.3 (July 2003), pp. 1162–1176. DOI: 10.1104/pp.102.017715.

[10] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[11] Zhihan Zhou et al. "DNABERT-2: Efficient foundation model and benchmark for multi-species genomes". In: *International Conference on Learning Representations*. Vol. 2024. 2024, pp. 41642–41665.